

Article

HySV: A Hymba-Inspired Hybrid-Head Framework for Quality-Aware and Deployment-Aware Speaker Verification in Intelligent Embedded Systems

Sundareswari Thiagarajan  and Deok-Hwan Kim 

Department of Electrical and Computer Engineering, Inha University, Incheon 22212, Republic of Korea; sundari@inha.edu

* Correspondence: deokhwan@inha.ac.kr

Abstract

Speaker verification is an important biometric technology for secure and personalized human–computer interaction in intelligent embedded systems. However, deploying deep speaker verification models on edge devices remains challenging because of restricted computational resources and strict real-time latency requirements. Existing systems commonly rely on convolutional, time-delay, or Transformer-based encoders. Although ECAPA-TDNN-based models provide strong verification performance, their temporal modeling mainly depends on convolutional and TDNN-style operations. Transformer-based models can capture broader temporal patterns, but they often require high computational and memory costs, making them less suitable for embedded deployment. To address these limitations, this paper proposes HySV, a Hymba-inspired hybrid attention and state-space encoder for deployment-aware speaker verification. Rather than directly employing the original Hymba language model, HySV adapts its hybrid-head principle to speaker embedding extraction. Specifically, conventional ECAPA-TDNN-style encoder blocks are replaced with three stacked Hymba context blocks. Each block contains an attention branch for local speaker-discriminative cue modeling and a state-space branch for efficient temporal context summarization. In addition, a quality-aware decision support module is introduced after cosine similarity scoring to improve reliability using utterance duration, voice activity ratio, and embedding confidence. The proposed system is evaluated using both speaker verification and deployment-oriented metrics, including EER, minDCF, FLOPs, and latency.

Keywords: speaker verification; intelligent embedded systems; Hymba-inspired architecture; hybrid-head encoder; quality-aware decision support; state-space modeling; attention-based embeddings



Academic Editor: Silvia Liberata Ullo

Received: 26 May 2026

Revised: 11 June 2026

Accepted: 14 June 2026

Published: 17 June 2026

Copyright: © 2026 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

1. Introduction

Speaker verification enables identity authentication from speech and is widely used in voice assistants, smart devices, access control systems, and embedded edge platforms [1,2]. Compared with password- or token-based authentication, speaker verification offers a natural and contactless way to authenticate users in embedded systems.

Recent advances in deep speaker embedding learning have significantly improved speaker verification performance. Earlier speaker verification systems relied on statistical modeling methods such as i-vectors and probabilistic linear discriminant analysis [3]. Later, x-vector systems introduced time-delay neural network-based speaker embedding extraction and became a widely used baseline for text-independent speaker verification [4]. More

recent architectures, especially ECAPA-TDNN, improved speaker embedding quality by using emphasized channel attention, multi-scale temporal aggregation, feature propagation, and attentive statistics pooling [5]. Large-scale datasets such as VoxCeleb1 and VoxCeleb2 have further accelerated speaker verification research by providing diverse real-world speech data collected from unconstrained environments [6,7].

Although these methods achieve strong verification performance, deploying deep speaker verification models on embedded edge devices remains challenging because of limited memory, restricted computation, low energy budgets, and latency constraints [8]. Therefore, a speaker verification model for embedded deployment must not only achieve low Equal Error Rate (EER) and minimum Detection Cost Function (minDCF), but also maintain low model size, low floating-point operations, low inference latency, and low memory usage. A model that performs well on a server-side GPU may not be practical for real-time speaker authentication on embedded hardware.

Another challenge is reliable decision-making under real-world speech conditions. In practical embedded environments, the input utterance may be short, noisy, partially silent, or affected by microphone quality. Short utterances contain limited speaker-discriminative information and may reduce verification reliability [9]. Similarly, low voice activity or unstable acoustic quality can result in less reliable speaker embeddings. Most speaker verification systems make final decisions mainly using cosine similarity between enrollment and test embeddings. However, cosine similarity alone may be insufficient when the input speech quality is poor. Therefore, a practical embedded speaker verification system should consider not only speaker similarity but also utterance-level quality indicators.

Efficient sequence modeling offers new opportunities for speaker verification. Transformer-based models capture long-range temporal dependencies but often require high computational and memory costs. State-space models such as Mamba provide efficient temporal context summarization with lower complexity [10]. Hymba further combines attention and state-space heads, where attention supports detailed information recall and state-space modeling supports efficient context summarization [11].

Motivated by these challenges, this paper proposes HySV, a Hymba-inspired hybrid-head framework for quality-aware and deployment-aware speaker verification in intelligent embedded systems. HySV reformulates the hybrid attention and state-space principle for speaker embedding extraction by assigning complementary roles to two branches. The attention branch captures local speaker-discriminative cues, while the state-space branch efficiently summarizes longer temporal speech context. Unlike ECAPA-TDNN, which mainly relies on convolutional and TDNN-style temporal modeling, and Mamba-style models, which mainly depend on state-space sequence modeling, HySV combines local cue extraction and temporal context summarization within a unified speaker encoder. Compared with the original Hymba design, HySV adapts the hybrid-head principle to speaker embedding extraction, attentive statistics pooling, cosine similarity scoring, and quality-aware score refinement. To improve verification under low-quality speech conditions, HySV also introduces a lightweight quality-aware decision support module after cosine similarity scoring.

The proposed framework is evaluated in terms of verification performance and deployment suitability using EER, minDCF, FLOPs, and latency. Experimental results show that HySV achieves improved performance compared with ResNet and ECAPA-TDNN baselines, with HySV ($C = 1024$) achieving an EER of 0.841% and a DCF of 0.1107.

2. Related Work

2.1. Deep Speaker Verification

Speaker verification has evolved from traditional statistical modeling approaches to deep speaker embedding learning. Early systems commonly used i-vector representations with probabilistic linear discriminant analysis for speaker verification [3]. These methods were effective under many benchmark conditions, but their performance was often limited in highly variable real-world acoustic environments.

Deep neural speaker embedding models significantly improved speaker verification performance. The x-vector framework introduced TDNN-based speaker embedding extraction and became a standard baseline for text-independent speaker verification [4]. In this approach, frame-level acoustic features are processed by temporal neural layers and aggregated into fixed-dimensional speaker embeddings. ECAPA-TDNN further improved speaker verification by using emphasized channel attention, Res2Net-style multi-scale temporal modeling, feature propagation, and attentive statistics pooling [5]. Because of its strong performance and relatively efficient architecture, ECAPA-TDNN is widely used as a baseline in modern speaker verification research.

However, ECAPA-TDNN mainly relies on convolutional and TDNN-style temporal modeling. These operations are efficient and effective for local and medium-range speech patterns, but they may not fully capture long-range temporal dependencies distributed across an utterance. This limitation motivates the exploration of hybrid temporal modeling approaches that preserve computational efficiency while improving long-context speaker representation.

2.2. Attention-Based Speaker Embeddings

Attention mechanisms have been widely used in speaker verification to improve utterance-level representation learning. Attentive statistics pooling assigns different weights to frame-level features, allowing the model to focus more strongly on speaker-informative frames [12]. This is important because not all frames contribute equally to speaker identity. Voiced frames, stable phonetic regions, and high-quality speech segments usually contain more speaker-discriminative information than silence, noise, or unstable frames.

Transformer-style self-attention can model long-range dependencies by allowing each frame to interact with other frames in the sequence [13]. However, full self-attention has quadratic computational complexity with respect to sequence length. For long speech sequences, this can increase memory usage and inference latency, making pure Transformer-based encoders less suitable for embedded speaker verification. Therefore, attention should be used carefully in deployment-aware systems. A lightweight attention branch can capture useful local speaker cues, while another efficient temporal mechanism can summarize a longer speech context.

2.3. State-Space Models and Efficient Sequence Modeling

State-space models have recently gained attention for efficient long-sequence modeling. Mamba introduced selective state-space modeling for linear-time sequence modeling and efficient handling of long contexts [10]. Unlike full self-attention, which explicitly compares tokens or frames across a sequence, state-space models summarize temporal information through structured state representations. This makes them attractive for long temporal signals such as speech.

Recent state-space and Mamba-based models have shown increasing potential for efficient speech and audio sequence modeling. Audio Mamba introduced selective state-space modeling for self-supervised audio representation learning, showing that Mamba-style

architectures can be effectively adapted to audio representation tasks [14]. Speech-Mamba further explored selective state-space models for long-context speech recognition, demonstrating their ability to model extended temporal dependencies in speech sequences [15]. Similarly, SSAMBA applied Mamba state-space modeling to self-supervised audio representation learning, supporting the relevance of SSM-based architectures for audio feature extraction [16]. These studies suggest that state-space models are becoming important alternatives to attention-based architectures for efficient speech and audio modeling. However, most existing SSM-based speech studies focus mainly on representation learning or recognition tasks, while their integration with quality-aware score refinement and deployment-aware speaker verification remains less explored.

However, pure state-space models may have weaker high-resolution recall compared with attention-based models. Hymba addresses this issue by combining attention heads and state-space model heads in a parallel hybrid-head module [11]. In this design, attention heads provide high-resolution recall, while state-space model heads provide efficient context summarization. The Hymba paper interprets attention as a form of snapshot memory and state-space modeling as a form of fading memory. Attention can preserve detailed information from specific positions, while state-space modeling summarizes long-range context efficiently.

2.4. Embedded Speaker Verification

Embedded speaker verification requires models that are accurate, compact, and efficient. Unlike server-side systems, embedded devices usually have limited memory, limited computational resources, and strict latency requirements [8]. Therefore, evaluating only EER and minDCF is not sufficient for embedded deployment. Practical deployment also requires metrics such as parameter count, FLOPs, embedding extraction latency, pairwise verification latency.

Many speaker verification studies focus mainly on improving verification accuracy, while deployment-oriented evaluation receives less attention. However, for intelligent embedded systems such as smart speakers, mobile devices, robots, access control terminals, and in-vehicle assistants, real-time response and low resource usage are essential. A model with slightly better verification accuracy may not be practical if it requires excessive memory, high latency, or high computational cost.

The proposed HySV model is designed for this setting. It replaces conventional ECAPA-TDNN-style encoder blocks with three stacked Hymba context blocks while keeping the remaining speaker verification pipeline compatible with standard speaker embedding systems. The use of Conv1D projection, attentive statistics pooling, FC projection, L2 normalization, and cosine similarity provides a practical structure for embedded deployment.

3. Methodology

3.1. Overview of the Proposed HySV Model

The proposed HySV model is designed as a Hymba-inspired hybrid encoder for deployment-aware speaker verification in intelligent embedded systems. The proposed model adapts the hybrid attention–state-space design principle of Hymba to speaker embedding extraction. The main objective of the proposed model is to improve temporal speaker representation while maintaining computational efficiency for embedded deployment.

The proposed architecture follows a standard speaker verification structure consisting of feature extraction, frame-level encoding, utterance-level pooling, speaker embedding extraction, similarity scoring, and final decision-making, as illustrated in Figure 1. Each Hymba context block contains a HySV encoder block, where an attention branch and a

state-space model branch operate in parallel. The attention branch captures local speaker-discriminative cues, while the state-space branch efficiently summarizes temporal speech context. The outputs of both branches are combined using gating and normalization to generate a unified speaker representation.

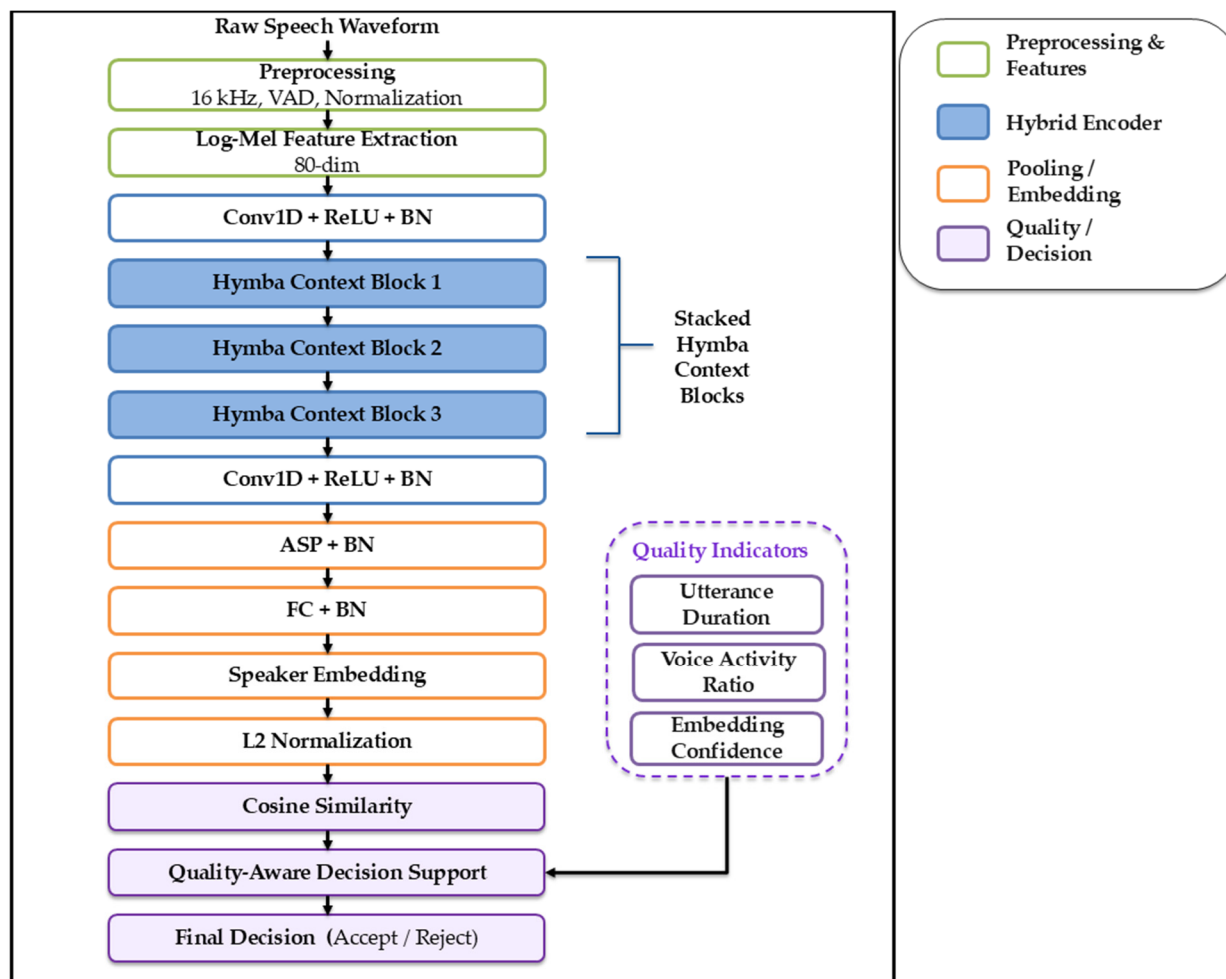


Figure 1. Overall architecture of the proposed HySV system with stacked Hymba context blocks and quality-aware decision support.

After the three stacked Hymba context blocks, a second one-dimensional convolution layer with Rectified Linear Unit activation and batch normalization is applied to refine the encoded speaker features before pooling. Attentive statistics pooling is then used to aggregate frame-level features into an utterance-level representation by computing the weighted mean and weighted standard deviation across time. This representation is passed through a fully connected layer with batch normalization to generate the speaker embedding, followed by L2 normalization. During verification, the enrollment and test embeddings are compared using cosine similarity. Finally, a quality-aware decision support module refines the cosine similarity score using utterance duration, voice activity ratio, and embedding confidence before producing the final accept/reject decision.

3.2. Feature Extraction and Front-End Processing

The first stage of the proposed HySV model is feature extraction, where the raw speech waveform is converted into a speaker-discriminative acoustic representation. The input speech signal is first resampled to 16 kHz to maintain a consistent sampling rate across all utterances. Voice activity detection is then applied to reduce silent and non-speech regions,

allowing the model to focus on speech segments that contain speaker identity information. After voice activity detection, waveform normalization is performed to reduce amplitude variations caused by different microphones, recording environments, and input levels.

After preprocessing, the speech signal is converted into 80-dimensional Log-Mel filterbank features. Log-Mel features are widely used in speaker verification because they represent short-time spectral characteristics of speech on a perceptually meaningful frequency scale. These features preserve important speaker-related information such as vocal tract characteristics, spectral shape, phonetic variation, and energy distribution.

Let the extracted Log-Mel feature sequence [4,5] be represented as

$$X = \{x_1, x_2, \dots, x_T\}, x_t \in \mathbb{R}^{80} \quad (1)$$

where T denotes the number of speech frames and x_t represents the 80-dimensional Log-Mel feature vector at frame t .

The extracted features are then passed through an initial one-dimensional convolution layer with Rectified Linear Unit activation and batch normalization. This front-end layer projects the acoustic features into a higher-dimensional frame-level representation and prepares them for the proposed Hymba context blocks. The one-dimensional convolution captures short-range temporal patterns from neighboring speech frames, the Rectified Linear Unit introduces non-linearity, and batch normalization stabilizes the feature distribution during training.

The front-end projection [4,5] is expressed as

$$H_0 = \text{BN}(\text{ReLU}(\text{Conv1D}(X))) \quad (2)$$

$$H_0 \in \mathbb{R}^{B \times T \times C} \quad (3)$$

where H_0 is the projected frame-level speech representation.

Here, B denotes the batch size, T denotes the number of frames, and C denotes the projected feature dimension. The resulting representation H_0 is then used as the input to the three stacked Hymba context blocks.

3.3. HySV Encoder Block

The HySV encoder block is the main component of the proposed architecture, as illustrated in Figure 2. It is inspired by the hybrid-head structure of Hymba, where attention-based modeling and state-space modeling are combined to process sequential information. In the proposed speaker verification system, this concept is adapted to speech by using an attention branch for local speaker cue extraction and a state-space branch for efficient temporal context modeling.

Given an input feature sequence

$$H \in \mathbb{R}^{B \times T \times C} \quad (4)$$

the input is first processed using layer normalization and Rectified Linear Unit activation [11,13]:

$$\tilde{H} = \text{ReLU}(\text{LayerNorm}(H)) \quad (5)$$

where H denotes the input feature sequence to the HySV encoder block, $\tilde{H}$ denotes the normalized and activated feature representation, $\text{LayerNorm}(\cdot)$ denotes layer normalization applied along the feature dimension, and $\text{ReLU}(\cdot)$ denotes the Rectified Linear Unit activation function.

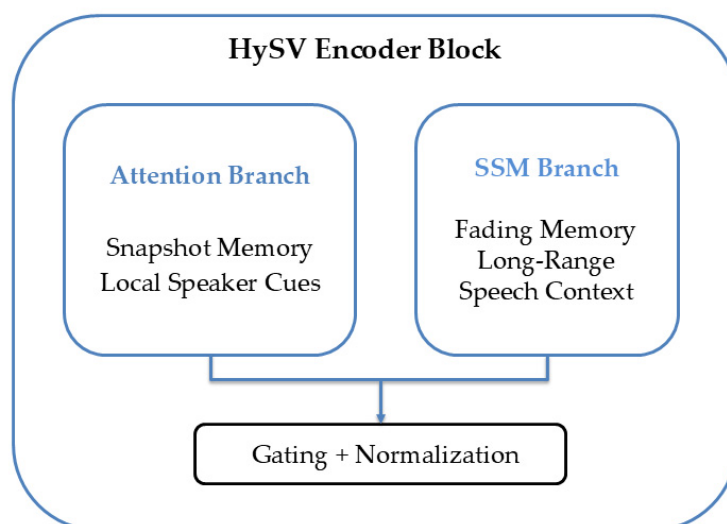


Figure 2. HySV encoder block with parallel attention and state-space branches.

The normalized feature sequence is then processed by the [11,13] attention branch:

$$H_{\text{att}} = f_{\text{att}}(\tilde{H}) \quad (6)$$

where f_{att} denotes the attention-based temporal modeling function. This branch captures local speaker-discriminative information such as short-time spectral patterns, voiced-frame characteristics, and local phonetic transitions.

State-space modeling is used to summarize temporal speech context efficiently, following Mamba- and Hymba-style state-space sequence modeling principles [10,11]:

$$H_{\text{ssm}} = f_{\text{ssm}}(\tilde{H}) \quad (7)$$

where f_{ssm} denotes the state-space sequence modeling function and H_{ssm} represents the temporal context representation generated by the state-space branch.

In this work, f_{ssm} is designed based on the selective state-space modeling principles used in Mamba- and Hymba-style architectures [10,11]. The normalized input sequence $\tilde{H}$ is first projected into an internal representation and then processed along the temporal dimension. The state-space branch uses learnable projection parameters and input-dependent temporal parameters to control how information is retained, updated, or suppressed across frames. This selective mechanism allows the model to preserve speaker-relevant temporal information while reducing the complexity associated with full self-attention.

The outputs of the two branches are combined using fusion/gating and normalization [11]:

$$H_{\text{fused}} = \text{Norm}(g_{\text{att}} \odot H_{\text{att}} + g_{\text{ssm}} \odot H_{\text{ssm}}) \quad (8)$$

where g_{att} and g_{ssm} are input-dependent gating weights for the attention and state-space branches, respectively, and $\odot$ denotes element-wise multiplication. Following hybrid-head fusion principles used in Hymba-style architectures [11], the gating weights are generated from the branch representations using learnable projection layers and normalized to control the relative contribution of each branch. This allows the encoder to adaptively balance local attention-based speaker cues and longer state-space temporal context during fusion.

The fused representation is then [5,8,11] projected and added to the original input using a residual connection:

$$H_{\text{out}} = H + W_{\text{proj}} H_{\text{fused}} \quad (9)$$

where W_{proj} denotes the learnable projection matrix used to map the fused representation back to the input feature dimension before residual addition, and H_{out} denotes the output of the HySV encoder block. This structure allows the HySV encoder block to combine attention-based local speaker cue modeling and state-space-based temporal summarization within a single speaker representation block.

3.4. Hymba Context Block

The Hymba context block is introduced to replace the conventional ECAPA-TDNN-style encoder block. Each Hymba context block consists of a one-dimensional convolution layer with Rectified Linear Unit activation and batch normalization, the HySV encoder block, fusion and normalization, a second one-dimensional convolution layer with Rectified Linear Unit activation and batch normalization, a squeeze-and-excitation block, and a residual connection.

The first one-dimensional convolution layer [5] projects the input representation into an internal feature space:

$$H_p = \text{BN}(\text{ReLU}(\text{Conv1D}(H))) \quad (10)$$

The projected representation is then processed by the HySV encoder block [10,11]:

$$H_e = f_{\text{HySV}}(H_p) \quad (11)$$

After the hybrid attention and state-space processing, a [5] second one-dimensional convolution layer is applied to refine the encoded representation:

$$H_c = \text{BN}(\text{ReLU}(\text{Conv1D}(H_e))) \quad (12)$$

The squeeze-and-excitation block [5] recalibrates the channel-wise speaker features:

$$H_{\text{se}} = f_{\text{SE}}(H_c) \quad (13)$$

Finally, a residual connection is applied to preserve the original input information and improve gradient flow [5,8]:

$$H_{\text{out}} = H + H_{\text{se}} \quad (14)$$

Here, H_p , H_e , H_c , and H_{se} denote the projected feature representation, HySV-encoded representation, convolution-refined representation, and squeeze-and-excitation-recalibrated representation, respectively. $f_{\text{HySV}}(\cdot)$, $f_{\text{SE}}(\cdot)$, and $f_{\text{HCB}}(\cdot)$ denote the HySV encoder block, squeeze-and-excitation block, and Hymba context block functions, respectively.

In the proposed architecture, three Hymba context blocks [5,11] are stacked sequentially:

$$H_1 = f_{\text{HCB1}}(H_0), \quad (15)$$

$$H_2 = f_{\text{HCB2}}(H_1), \quad (16)$$

$$H_3 = f_{\text{HCB3}}(H_2). \quad (17)$$

Here, H_0 denotes the representation input into the stacked Hymba context blocks. H_1 , H_2 , and H_3 denote the output representations of the first, second, and third Hymba context blocks, respectively. $f_{\text{HCB1}}(\cdot)$, $f_{\text{HCB2}}(\cdot)$, and $f_{\text{HCB3}}(\cdot)$ represent the transformation functions of the three successive Hymba context blocks. Each block applies hybrid attention-state-space modeling to progressively refine the frame-level speaker representation.

After the three stacked Hymba context blocks, a post-context one-dimensional convolution layer with [5] Rectified Linear Unit activation and batch normalization is applied:

$$H_f = \text{BN}(\text{ReLU}(\text{Conv1D}(H_3))) \quad (18)$$

where H_f denotes the final frame-level feature representation after post-context convolutional refinement. This layer further refines the speaker-discriminative frame-level features before attentive statistics pooling.

3.5. Attentive Statistics Pooling

After frame-level encoding, attentive statistics pooling is used to convert variable-length frame-level features into a fixed-dimensional utterance-level representation. This pooling mechanism assigns different weights to different frames, allowing the model to emphasize speaker-informative regions.

For each frame t , an [12] attention weight α_t is computed as

$$\alpha_t = \frac{\exp(e_t)}{\sum_{j=1}^T \exp(e_j)} \quad (19)$$

The weighted mean [12] is calculated as

$$\mu = \sum_{t=1}^T \alpha_t h_t \quad (20)$$

where h_t denotes the frame-level encoded feature at time frame t , and e_t denotes the scalar attention score assigned to frame t . The attention weight α_t is obtained by applying the softmax function over all frame-level attention scores.

The weighted standard deviation [12] is calculated as

$$\sigma = \sqrt{\sum_{t=1}^T \alpha_t h_t^2 - \mu^2} \quad (21)$$

The final utterance-level representation [12] is obtained by concatenating the weighted mean and weighted standard deviation:

$$z = [\mu; \sigma] \quad (22)$$

where μ and σ denote the attention-weighted mean and standard deviation vectors of the frame-level features, respectively. The notation $[\cdot]$ indicates vector concatenation, and z represents the pooled utterance-level speaker representation.

This representation captures both the average speaker characteristics and the temporal variability of the encoded speech features.

3.6. Speaker Embedding Extraction and L2 Normalization

The [4,5] utterance-level representation obtained from attentive statistics pooling is passed through a fully connected layer with batch normalization:

$$e = \text{BN}(\text{FC}(z)) \quad (23)$$

The vector e represents the extracted speaker embedding. To make similarity scoring stable and scale-invariant, L2 normalization [5,17] is applied:

$$\hat{e} = \frac{e}{\|e\|_2} \quad (24)$$

The normalized speaker embedding $\hat{e}$ is used for enrollment–test comparison during speaker verification.

3.7. Training Objective

The proposed model can be trained using Additive Angular Margin Softmax [17,18], which improves speaker embedding discrimination by increasing inter-speaker separation and intra-speaker compactness in the embedding space.

The loss is defined as

$$\mathcal{L}_{AAM} = -\frac{1}{N} \sum_{i=1}^N \log \frac{e^{s \cdot \cos(\theta_{y_i} + m)}}{e^{s \cdot \cos(\theta_{y_i} + m)} + \sum_{j \neq y_i} e^{s \cdot \cos(\theta_j)}} \quad (25)$$

where N is the batch size, s is the scale factor, m is the angular margin, and y_i is the ground-truth speaker label. During inference, the classification layer is removed, and the normalized speaker embedding is used for cosine similarity scoring.

3.8. Cosine Similarity Scoring

During verification, both enrollment and test utterances are processed by the same embedding network. Let the enrollment embedding be $\hat{e}_{enroll}$ and the test embedding be $\hat{e}_{test}$. The cosine similarity score [2,5] is computed as

$$S_{\cos} = \frac{\hat{e}_{enroll} \cdot \hat{e}_{test}}{\|\hat{e}_{enroll}\|_2 \|\hat{e}_{test}\|_2} \quad (26)$$

Since both embeddings are [5] L2-normalized, the cosine similarity score becomes

$$S_{\cos} = \hat{e}_{enroll} \cdot \hat{e}_{test} \quad (27)$$

A higher cosine similarity score indicates a higher probability that the enrollment and test utterances belong to the same speaker.

3.9. Quality-Aware Decision Support

Cosine similarity [19] alone may be unreliable when the input speech is short, noisy, or contains low speech activity. Therefore, the proposed HySV model includes a lightweight quality-aware decision support module after cosine similarity scoring. This module uses utterance duration, voice activity ratio, and embedding confidence as low-cost quality indicators.

The quality vector is defined as

$$q = [q_{dur}, q_{vad}, q_{conf}], \quad (28)$$

where q_{dur} denotes utterance duration, q_{vad} denotes voice activity ratio, and q_{conf} denotes embedding confidence. Embedding confidence is defined as a lightweight estimate of the stability of the enrollment–query embedding comparison. Since the embeddings are L2-normalized before cosine scoring, the confidence is estimated from the cosine similarity score relative to the decision threshold. A score farther from the decision threshold is treated as more confident, while a score closer to the threshold is treated as less confident.

This confidence value is combined with utterance duration and voice activity ratio to form the quality vector used for quality-aware score refinement.

A quality-aware scaling weight is calculated as

$$w_q = \sigma(W_q q + b_q), \quad (29)$$

where W_q and b_q are learnable parameters, and $\sigma(\cdot)$ is the sigmoid function.

The final quality-refined score is calculated as

$$S_{\text{final}} = w_q \cdot S_{\text{cos}}. \quad (30)$$

where S_{cos} denotes the original cosine similarity score between the enrollment and query speaker embeddings, w_q denotes the quality-aware weighting factor estimated from utterance-level quality indicators, and S_{final} denotes the quality-refined verification score.

The final verification decision is made using a [1,2] threshold θ :

$$\text{Decision} = \begin{cases} \text{Accept}, & S_{\text{final}} \geq \theta, \\ \text{Reject}, & S_{\text{final}} < \theta. \end{cases} \quad (31)$$

This module improves score refinement by reducing overconfident accept/reject decisions for short-duration, low-activity, or low-confidence speech inputs. The quality-aware decision module uses utterance duration, voice activity ratio, and embedding confidence because these indicators reflect common sources of unreliable speaker verification decisions while requiring minimal computational overhead. Utterance duration represents the amount of available speech evidence; very short utterances often contain insufficient speaker-discriminative information. The voice activity ratio measures the proportion of useful speech frames after voice activity detection and helps identify inputs dominated by silence or non-speech regions. Embedding confidence reflects the stability of the extracted speaker representation before final scoring. These indicators were selected because they could be computed without an additional heavy quality-estimation network, making them suitable for embedded and resource-constrained deployment.

3.10. Equal Error Rate Evaluation

The verification performance of the proposed system is evaluated using Equal Error Rate. Equal Error Rate is the point at which the false acceptance rate and false rejection rate are equal. The false acceptance rate measures the proportion of non-target speaker trials that are incorrectly accepted, while the false rejection rate measures the proportion of target speaker trials that are incorrectly rejected.

For a given decision [1,2] threshold θ , the false acceptance rate is defined as

$$\text{FAR}(\theta) = \frac{\text{Number of false accepts}}{\text{Number of non-target trials}} \quad (32)$$

The false rejection rate [1,2] is defined as

$$\text{FRR}(\theta) = \frac{\text{Number of false rejects}}{\text{Number of target trials}}. \quad (33)$$

The Equal Error Rate [1,2] is obtained when

$$\text{FAR}(\theta) = \text{FRR}(\theta). \quad (34)$$

A lower Equal Error Rate indicates better speaker verification performance.

4. Experimental Setup

4.1. Dataset

The proposed HySV model is evaluated using the VoxCeleb [6] speaker verification protocol. VoxCeleb is a large-scale speaker recognition dataset collected from real-world audio-visual sources and contains speech from diverse speakers, accents, acoustic conditions, and background environments. In this work, the development part of VoxCeleb2 [7] was used for model training, containing 5994 speakers. Approximately 2% of the training data was reserved as a validation set for hyperparameter tuning, best-checkpoint selection, and decision-threshold estimation. Final speaker verification evaluation was performed using the official VoxCeleb1 cleaned verification trial list, `veri_test2.txt`.

All audio recordings were converted to single-channel waveforms and resampled to 16 kHz. Voice activity detection was applied to remove silence and non-speech regions, followed by waveform amplitude normalization. Each utterance was converted into 80-dimensional Log-Mel filterbank features using a 25 ms analysis window and a 10 ms frame shift. During training, fixed-length 3 s speech segments were randomly sampled from each utterance to form mini-batches. During evaluation, speaker embeddings were extracted from the enrollment and test utterances specified in `veri_test2.txt`, and cosine similarity scores were computed for all verification trials. The final performance is reported using EER and minDCF.

4.2. Baseline Models

To evaluate the effectiveness of the proposed HySV encoder, several baseline systems were considered [5,7,20]. The ECAPA-TDNN model was used as the main baseline because the proposed model replaces conventional ECAPA-TDNN-style encoder blocks with three stacked Hymba context blocks. Additional baselines were included to compare the proposed model with traditional deep speaker embedding systems and efficient sequence modeling approaches.

4.3. Proposed Model Configuration

The proposed HySV model uses 80-dimensional Log-Mel filterbank features as input. These features are first processed using a one-dimensional convolution layer with Rectified Linear Unit activation and batch normalization. The resulting frame-level representation is passed through three stacked Hymba context blocks. Each Hymba context block contains a HySV encoder block with an attention branch and a state-space model branch. The outputs of the two branches are combined using fusion/gating and normalization.

After the three Hymba context blocks, another one-dimensional convolution layer with Rectified Linear Unit activation and batch normalization is used to refine the encoded features. Attentive statistics pooling is then applied to generate an utterance-level representation. A fully connected layer with batch normalization produces the speaker embedding, followed by L2 normalization. During inference, enrollment and test embeddings are compared using cosine similarity, and the final decision is refined by the quality-aware decision support module.

4.4. Training Setup

The model was trained using speaker labels from the training set. During training, each utterance was converted into Log-Mel filterbank features and fixed-length segments were randomly sampled to form mini-batches. The network was optimized using Additive Angular Margin Softmax, which improves speaker discrimination by increasing inter-speaker separation and reducing intra-speaker variation in the embedding space.

The validation subset was used to select the best checkpoint and estimate the decision threshold θ . Validation cosine similarity scores were computed from target and non-target validation trials, and θ was selected based on the validation-set operating point. After the threshold was fixed, the final evaluation was performed on the official VoxCeleb1 cleaned trial list, `veri_test2.txt`. No threshold tuning or model selection was performed on the VoxCeleb1 evaluation trials.

For model selection, approximately 2% of the VoxCeleb2 development data was held out as a validation set and was not used for parameter optimization. This validation subset was used for hyperparameter tuning, decision-threshold estimation, and best-checkpoint selection. The model was trained for 80 epochs using Additive Angular Margin Softmax as the training objective. The best checkpoint was selected based on the lowest validation EER. Early stopping was applied when the validation EER did not improve for a fixed patience period. During final evaluation, the selected checkpoint and fixed threshold were applied to the official VoxCeleb1 cleaned verification trial list, `'veri_test2.txt'`, without further tuning.

Data augmentation can be applied to improve robustness under real-world acoustic conditions. Additive noise, music, speech interference, and room impulse response-based reverberation can be used during training. These augmentations simulate noisy and reverberant environments commonly encountered in embedded speaker verification applications. Table 1 summarizes the training configuration of the proposed HySV architecture.

Table 1. Training configuration.

Item	Setting
Input sampling rate	16 kHz
Acoustic feature	80-dimensional Log-Mel filterbank
Segment duration	2–3 s
Training objective	Additive Angular Margin Softmax
Optimizer	Adam
Augmentation	Noise, music, speech interference, reverberation
Embedding scoring	Cosine similarity

4.5. Evaluation Protocol

The trained model was evaluated using speaker verification trials. For each trial, the enrollment utterance and test utterance were passed through the same HySV embedding network. The extracted embeddings were L2-normalized and compared using cosine similarity. Based on the similarity score, the system decided whether the two utterances belonged to the same speaker.

The primary speaker verification metrics were Equal Error Rate and minimum Detection Cost Function. Equal Error Rate is the point where the false acceptance rate and false rejection rate are equal. A lower Equal Error Rate indicates better verification performance. The minimum Detection Cost Function measures the cost of speaker verification decisions under predefined target and non-target conditions.

4.6. Embedded Deployment Evaluation

Since the proposed model was designed for intelligent embedded systems, deployment-related metrics were also evaluated. These included number of parameters, floating-point operations, embedding extraction latency, and pairwise verification latency.

Embedding latency refers to the time required to extract one speaker embedding from an input utterance. Pairwise verification latency includes the time needed to extract

embeddings from both enrollment and test utterances and compute their cosine similarity. It can be approximated as

$$Latency_{pair} \approx 2 \times Latency_{embedding} \quad (35)$$

4.7. Hardware and Profiling Setup

All models were profiled under the same input duration, feature dimension, batch size, and hardware configuration to ensure a fair comparison. The embedded profiling was conducted on an NVIDIA Jetson AGX Xavier Developer Kit, model P2972 (NVIDIA Corporation, Santa Clara, CA, USA). The experiments were implemented using Python 3.8 and PyTorch 2.1.0 with an FP32 inference pipeline. The batch size was set to 1, and the input consisted of an enrollment–query utterance pair. The Jetson AGX Xavier platform was selected because it is a commonly used embedded AI development platform for evaluating deep learning models under edge-computing and resource-constrained deployment conditions.

Latency was measured after warm-up runs and averaged over multiple inference trials. The reported latency values are measured in seconds and correspond to the complete current inference pipeline, including feature processing, embedding extraction, and pairwise verification scoring.

5. Results

5.1. Speaker Verification Performance

Table 2 presents the speaker verification performance of the proposed HySV model compared with ResNet and ECAPA-TDNN baseline systems. The performance is evaluated using Equal Error Rate (EER) and Detection Cost Function (DCF). Lower values of both metrics indicate better speaker verification performance.

Table 2. Speaker verification performance comparison.

Model	EER (%)	DCF
ResNet18	1.610	0.1989
ResNet34	1.264	0.1341
ECAPA (C = 512)	1.158	0.1221
ECAPA (C = 1024)	0.956	0.1266
HySV (C = 512)	0.850	0.1168
HySV (C = 1024)	0.841	0.1107

The proposed HySV models achieve the best overall verification performance among all compared systems. HySV with channel size 512 obtains an EER of 0.850% and a minDCF of 0.1168, while HySV with channel size 1024 achieves the best result with an EER of 0.841% and a minDCF of 0.1107. Compared with ECAPA (C = 512), the full HySV model (C = 1024) reduces the EER from 1.158% to 0.841%, showing a relative improvement of approximately 27.37%. The minDCF is also reduced from 0.1221 to 0.1107, showing improved score refinement.

Compared with ECAPA (C = 1024), HySV (C = 1024) reduces the EER from 0.956% to 0.841%, giving a relative improvement of approximately 12.03%. This indicates that the proposed Hymba-inspired hybrid attention–state-space encoder improves speaker representation beyond simply increasing the channel size. The results suggest that combining local speaker cue modeling through the attention branch and temporal context summarization through the state-space branch is effective for speaker verification.

5.2. Ablation Study

Table 3 presents a component-wise ablation study of the proposed HySV model. The purpose of this analysis was to examine the contribution of the attention branch, state-space branch, hybrid encoder design, and quality-aware decision support module.

Table 3. Ablation study of HySV components.

Model Variant	Attention Branch	State-Space Branch	Quality-Aware Module	EER (%)	95% CI for EER	minDCF
ECAPA-TDNN baseline	No	No	No	1.158	1.120–1.196	0.1221
Attention-only encoder	Yes	No	No	0.931	0.902–0.960	0.0976
State-space-only encoder	No	Yes	No	0.846	0.841–0.871	0.0904
Hybrid encoder	Yes	Yes	No	0.851	0.845–0.877	0.1179
Full HySV	Yes	Yes	Yes	0.841	0.821–0.866	0.1107

The ablation results show that replacing the ECAPA-TDNN encoder with the proposed Hymba-inspired components improves verification performance. The attention-only encoder reduces the EER from 1.158% to 0.931%, showing that attention-based local speaker cue modeling improves speaker-discriminative feature extraction. The state-space-only encoder further reduces the EER to 0.846%, indicating that state-space temporal modeling is highly effective for summarizing speech context.

The hybrid encoder without the quality-aware module obtains an EER of 0.851%, which is close to the state-space-only encoder. This suggests that hybrid fusion alone may not consistently reduce detection cost without quality-aware score refinement. However, its minDCF is higher than the state-space-only variant. After adding the quality-aware decision support module, the full HySV model achieves the best EER of 0.841% and a minDCF of 0.1107. This confirms that quality-aware decision support improves the final verification reliability.

To provide a more cautious statistical interpretation of the ablation results, 95% confidence intervals were added for the EER values. Full HySV obtains the lowest EER among the evaluated variants; however, the confidence intervals of the state-space-only encoder, hybrid encoder, and Full HySV overlap. This indicates that the performance differences among these closely performing variants are small and should be interpreted as empirical trends rather than definitive statistically significant improvements. Therefore, the results suggest a favorable overall trend for HySV, while further repeated trials and statistical testing are needed for stronger significance validation.

Compared with the ECAPA-TDNN baseline, the full HySV model reduces EER by approximately 27.37%. This demonstrates that the proposed hybrid attention–state-space encoder and quality-aware decision module provide a meaningful improvement over the conventional ECAPA-TDNN-style baseline.

Table 4, presenting the results of quality-aware indicator ablation, shows that adding utterance-level quality information gradually improves the final verification performance. The hybrid encoder without the quality-aware module obtains an EER of 0.851% and a minDCF of 0.1179. Adding utterance duration information slightly reduces both EER and minDCF, while combining duration with VAD ratio provides further improvement. The full quality-aware module, which uses duration, VAD ratio, and embedding confidence, achieves the best result with an EER of 0.841% and a minDCF of 0.1107. These results suggest that the quality-aware module provides a gradual score refinement effect.

Table 4. Ablation study of quality-aware score refinement indicators.

Model Variant	Duration	VAD Ratio	Embedding Confidence	EER (%)	minDCF
Hybrid encoder without quality module	No	No	No	0.851	0.1179
HySV + duration	Yes	No	No	0.848	0.1156
HySV + duration + VAD ratio	Yes	Yes	No	0.845	0.1138
Full HySV	Yes	Yes	Yes	0.841	0.1107

However, ROC-curve analysis and detailed evaluation under short-utterance and noisy conditions were not included in the current study. These analyses will be considered in future work to further validate the quality-aware module under more challenging speech conditions.

5.3. Latency and Embedded Deployment Analysis

Table 5 presents the latency comparison between the ECAPA-TDNN baseline, the base Hymba encoder, and the proposed HySV system on the NVIDIA Jetson platform. Latency is reported for both pairwise verification and single-utterance embedding extraction. Pair latency refers to the total time required to compare an enrollment utterance and a test utterance, while utterance latency represents the approximate time required to process one speech utterance.

Table 5. Latency comparison on the NVIDIA Jetson platform.

Model/Configuration	Latency/Pair	Latency/Utterance
ECAPA-TDNN Baseline	38–42 s	19–21 s
Base Hymba Encoder	35–37 s	17.5–18.5 s
HySV (Ours)	~33.77–34.60 s	≈16.9 s

Latency values are reported in seconds and correspond to the complete current inference pipeline. The results show that the proposed HySV system achieves the lowest inference latency among the compared configurations. The ECAPA-TDNN baseline requires approximately 38–42 s per verification pair and 19–21 s per utterance, indicating higher computational cost. The base Hymba encoder reduces the pair latency to 35–37 s, suggesting that the Hymba-inspired temporal encoder is more efficient than the ECAPA-TDNN baseline.

The complete HySV system further reduces the pair latency to approximately 33.77–34.60 s, with an estimated utterance latency of 16.9 s. Compared with the ECAPA-TDNN baseline, this represents a noticeable reduction in verification time. These results support the deployment-aware design of the proposed model by showing that HySV can reduce inference latency while maintaining strong verification performance.

5.4. Computational Complexity Analysis

Table 6 presents the computational complexity comparison of the proposed HySV model and baseline speaker verification systems. The comparison includes FLOPs, parameter count, EER, and DCF. FLOPs indicate the computational cost required during inference, while the number of parameters represents the model size and memory requirement. For deployment-aware speaker verification, the objective is not only to reduce EER and DCF, but also to maintain acceptable computational complexity for embedded platforms.

Table 6. Computational complexity comparison of speaker verification models.

Model/Configuration	FLOPs (G)	Params (M)	EER (%)	DCF
ResNet18	~7	~2.0	1.610	0.1989
ResNet34	~9.9	~3.7	1.264	0.1341
ECAPA-TDNN (C = 512)	~6.3	~2.6	1.158	0.1221
ECAPA-TDNN (C = 1024)	~16	~5.5	0.956	0.1266
HySV (C = 512)	~9.1	~3.0	0.850	0.1168
HySV (C = 1024)	~20	~5.7	0.841	0.1107

As shown in Table 6, ResNet18 has relatively low complexity, requiring approximately 7 GFLOPs and 2.0 M parameters. However, it produces the weakest verification performance, with an EER of 1.610% and a DCF of 0.1989. ResNet34 improves the EER to 1.264% and the DCF to 0.1341, but this improvement increases the computational cost to approximately 9.9 GFLOPs and 3.7 M parameters.

The ECAPA-TDNN (C = 512) baseline provides a strong balance between complexity and verification performance, requiring approximately 6.3 GFLOPs and 2.6 M parameters, with an EER of 1.158% and a DCF of 0.1221. Increasing the channel size to C = 1024 improves the EER to 0.956%, but the computational cost increases significantly to approximately 16 GFLOPs and 5.5 M parameters. This shows that higher channel capacity improves speaker representation but also increases deployment cost.

The proposed HySV (C = 512) achieves better verification performance than the ECAPA-TDNN baselines, with an EER of 0.850% and a DCF of 0.1168, while requiring approximately 9.1 GFLOPs and 3.0 M parameters. This indicates that the proposed hybrid-head encoder improves speaker-discriminative representation with acceptable computational complexity. The larger HySV (C = 1024) configuration achieves the best overall verification performance, with an EER of 0.841% and a DCF of 0.1107. Although it requires higher computational cost, approximately 20 GFLOPs and 5.7 M parameters, it remains deployment-aware because the improvement in verification performance is achieved with a controlled increase in model complexity.

Overall, the results show that HySV provides a better accuracy–complexity trade-off than the compared baselines. The HySV (C = 512) configuration is suitable when moderate computational cost is required, while HySV (C = 1024) is preferable when the best verification accuracy is the main objective. Therefore, the proposed HySV framework supports deployment-aware speaker verification by jointly considering verification accuracy, computational cost, and model size.

5.5. Discussion

The experimental results demonstrate that the proposed HySV framework improves speaker verification performance while considering deployment-related efficiency. In terms of verification accuracy, HySV (C = 1024) achieves the best performance among all compared models, with the lowest EER of 0.841% and DCF of 0.1107. Compared with ECAPA-TDNN (C = 512), the proposed model reduces the EER from 1.158% to 0.841%, showing an improvement of approximately 27.37%. Compared with ECAPA-TDNN (C = 1024), HySV (C = 1024) further reduces the EER from 0.956% to 0.841%, showing that the improvement is not only due to increasing channel size, but also due to the proposed hybrid-head encoder design.

The ablation study confirms the contribution of each component in the proposed architecture. The attention-only encoder improves the EER from 1.158% to 0.931%, indicating that attention-based local speaker cue modeling helps extract speaker-discriminative frame-level information. The state-space-only encoder achieves an EER of 0.846%, showing

that state-space temporal modeling is effective for summarizing speech context. The full HySV model, which combines the hybrid encoder with the quality-aware decision support module, achieves the best EER of 0.841%. This result suggests that the quality-aware module contributes to improving final verification reliability by using utterance duration, voice activity ratio, and embedding confidence.

The latency results also support the deployment-aware motivation of this work. On the NVIDIA Jetson platform, the ECAPA-TDNN baseline requires approximately 38–42 s per verification pair, while the base Hymba encoder reduces the latency to 35–37 s. The proposed HySV system further reduces the pairwise latency to approximately 33.77–34.60 s, with an estimated utterance latency of 16.9 s. This indicates that the proposed architecture reduces inference time compared with the ECAPA-TDNN baseline. Although further optimization is still needed for strict real-time deployment, the Jetson-based latency results show that HySV is more suitable for embedded speaker verification than the baseline configuration.

The computational complexity analysis further shows the trade-off between model performance and deployment cost. ResNet18 and ECAPA-TDNN ($C = 512$) have lower FLOPs and fewer parameters, but their EER and DCF are higher than those of HySV. In contrast, HySV ($C = 512$) achieves better verification performance with moderate complexity, requiring approximately 9.1 GFLOPs and 3.0 M parameters. HySV ($C = 1024$) achieves the best EER and DCF, requiring approximately 20 GFLOPs and 5.7 M parameters. This shows that the proposed model improves verification accuracy with a controlled increase in computational complexity. Therefore, HySV ($C = 512$) is suitable when moderate computational cost is preferred, while HySV ($C = 1024$) is suitable when the best verification performance is required.

The key distinction of the proposed HySV framework is its speaker-verification-specific reformulation of hybrid attention–state-space modeling rather than a simple combination of existing components. This design is important because speaker verification requires both fine-grained local speaker cues and longer temporal speech context. Local acoustic patterns, phonetic transitions, and voiced regions help distinguish speakers with similar global characteristics, while longer temporal context supports stable speaker representation across different phonetic content and utterance durations. ECAPA-TDNN-style models provide strong temporal aggregation through convolutional and TDNN-based operations, but they do not explicitly separate local cue modeling from longer context summarization. Mamba-style state-space models provide efficient sequence summarization, but using state-space modeling alone may reduce access to detailed local speaker-discriminative information. HySV addresses this gap by assigning complementary roles to attention and state-space branches inside stacked Hymba context blocks. This design is also important for embedded deployment because the state-space branch supports efficient temporal summarization, while the attention branch is used selectively to preserve local speaker cues instead of applying heavy attention across the entire sequence. The fused representation is then connected to attentive statistics pooling, L2-normalized speaker embedding extraction, cosine similarity scoring, and quality-aware score refinement. As a result, HySV provides a deployment-aware speaker verification framework that considers representation quality, and computational efficiency together.

Overall, the results indicate that HySV provides a better balance between speaker verification accuracy, computational complexity, and latency. The hybrid attention–state-space encoder improves speaker representation by combining local speaker cue modeling with temporal context summarization. The quality-aware decision support module further improves reliability by refining the final verification score. These findings support the

effectiveness of the proposed HySV framework for quality-aware and deployment-aware speaker verification in intelligent embedded systems.

Although HySV is designed for deployment-aware speaker verification, the current implementation should be interpreted as an initial deployment-aware system rather than a fully optimized real-time embedded solution. The latency values reported in this study are measured in seconds and reflect the complete current inference pipeline. The main objective of this work is to evaluate the proposed HySV architecture, including the hybrid attention–state-space encoder and quality-aware score refinement module, rather than to develop a hardware-specific optimized inference system. Hardware-specific optimization was not applied in this study because it requires a separate deployment optimization stage and may affect the accuracy–latency trade-off. Therefore, the reported latency should be interpreted as an initial deployment-aware baseline for the current implementation. Future work will focus on reducing latency for practical real-time embedded speaker verification through TensorRT conversion, quantization, pruning, operator fusion, streaming inference, and hardware-aware optimization.

6. Conclusions

This paper presented HySV, a Hymba-inspired hybrid-head framework for quality-aware and deployment-aware speaker verification in intelligent embedded systems. The proposed framework replaces conventional ECAPA-TDNN-style encoder blocks with three stacked Hymba context blocks. Each HySV encoder block combines an attention branch and a state-space branch, where the attention branch captures local speaker-discriminative cues and the state-space branch efficiently summarizes temporal speech context. In addition, a quality-aware decision support module was introduced after cosine similarity scoring to improve verification reliability using utterance duration, voice activity ratio, and embedding confidence.

The experimental results demonstrate that the proposed HySV model improves speaker verification performance compared with ResNet and ECAPA-TDNN baselines. HySV ($C = 1024$) achieved the best overall performance, with an EER of 0.841% and a DCF of 0.1107, outperforming ECAPA-TDNN ($C = 512$), which obtained an EER of 1.158% and a DCF of 0.1221. The ablation study further confirmed the effectiveness of the proposed components. The attention-only encoder improved local speaker cue modeling, while the state-space-only encoder provided strong temporal context summarization. The full HySV model achieved the best EER after incorporating the quality-aware decision support module.

The deployment analysis also supports the practical value of the proposed framework. On the NVIDIA Jetson platform, HySV achieved a pairwise verification latency of approximately 33.77–34.60 s and an utterance latency of approximately 16.9 s, which is lower than the ECAPA-TDNN baseline. The computational complexity analysis showed that HySV provides a better accuracy–complexity trade-off by improving verification performance while maintaining controlled FLOPs and parameter size. These results indicate that HySV provides a balanced solution for speaker verification accuracy, computational complexity, and embedded inference efficiency.

Future work will focus on further optimizing HySV for real-time edge deployment using model compression, quantization, TensorRT acceleration, and streaming inference. Additional experiments under noisy, short-duration, and low-quality speech conditions will also be conducted to further validate the robustness of the quality-aware decision support module.

Author Contributions: Conceptualization, S.T. and D.-H.K.; methodology, S.T. and D.-H.K.; software, S.T.; validation, S.T. and D.-H.K.; formal analysis, S.T.; investigation, S.T.; resources, D.-H.K.; data curation, S.T.; writing—original draft preparation, S.T.; writing—review and editing, S.T. and D.-H.K.; visualization, S.T.; supervision, D.-H.K.; project administration, D.-H.K.; funding acquisition, D.-H.K. All authors have read and agreed to the published version of the manuscript.

Funding: This research was supported in part by the National Research Foundation of Korea (NRF), funded by the Korean government (MSIT) under Grant No. RS-2024-00336286, and in part by the Inha University Research Grant.

Data Availability Statement: The speaker verification experiments were conducted using the publicly available VoxCeleb datasets. VoxCeleb1 and VoxCeleb2 can be accessed from the official project website: <https://www.robots.ox.ac.uk/~vgg/data/voxceleb/> (accessed on 20 May 2026).

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Reynolds, D.A. An overview of automatic speaker recognition technology. In *Proceedings of the IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP), Orlando, FL, USA, 13–17 May 2002*; IEEE: New York, NY, USA, 2002; Volume 4, pp. IV-4072–IV-4075.
2. Kinnunen, T.; Li, H. An overview of text-independent speaker recognition: From features to supervectors. *Speech Commun.* **2010**, *52*, 12–40. [\[CrossRef\]](#)
3. Dehak, N.; Kenny, P.J.; Dehak, R.; Dumouchel, P.; Ouellet, P. Front-end factor analysis for speaker verification. *IEEE Trans. Audio Speech Lang. Process.* **2011**, *19*, 788–798. [\[CrossRef\]](#)
4. Snyder, D.; Garcia-Romero, D.; Sell, G.; Povey, D.; Khudanpur, S. X-vectors: Robust DNN embeddings for speaker recognition. In *Proceedings of the IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP), Calgary, AB, Canada, 15–20 April 2018*; pp. 5329–5333.
5. Desplanques, B.; Thienpondt, J.; Demuynck, K. ECAPA-TDNN: Emphasized channel attention, propagation and aggregation in TDNN based speaker verification. In *Proceedings of the Interspeech 2020, Shanghai, China, 25–29 October 2020*; pp. 3830–3834.
6. Nagrani, A.; Chung, J.S.; Zisserman, A. VoxCeleb: A large-scale speaker identification dataset. In *Proceedings of the Interspeech 2017, Stockholm, Sweden, 20–24 August 2017*; pp. 2616–2620.
7. Chung, J.S.; Nagrani, A.; Zisserman, A. VoxCeleb2: Deep speaker recognition. In *Proceedings of the Interspeech 2018, Hyderabad, India, 2–6 September 2018*; pp. 1086–1090.
8. Sze, V.; Chen, Y.-H.; Yang, T.-J.; Emer, J.S. Efficient processing of deep neural networks: A tutorial and survey. *Proc. IEEE* **2017**, *105*, 2295–2329. [\[CrossRef\]](#)
9. Kanagasundaram, A.; Dean, D.; Sridharan, S.; Fookes, C.; Vogt, R. Improving short utterance i-vector speaker verification using utterance variance modelling and compensation techniques. *Speech Commun.* **2014**, *59*, 69–82. [\[CrossRef\]](#)
10. Gu, A.; Dao, T. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv* **2023**, arXiv:2312.00752.
11. Dong, X.; Fu, Y.; Diao, S.; Byeon, W.; Chen, Z.; Mahabaleshwarkar, A.S.; Liu, S.-Y.; Van Keirsbilck, M.; Chen, M.-H.; Suhara, Y.; et al. Hymba: A hybrid-head architecture for small language models. *arXiv* **2024**, arXiv:2411.13676.
12. Okabe, K.; Koshinaka, T.; Shinoda, K. Attentive statistics pooling for deep speaker embedding. In *Proceedings of the Interspeech 2018, Hyderabad, India, 2–6 September 2018*; pp. 2252–2256.
13. Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A.N.; Kaiser, Ł.; Polosukhin, I. Attention is all you need. In *Proceedings of the 31st International Conference on Neural Information Processing Systems (NeurIPS), Long Beach, CA, USA, 4–9 December 2017*; pp. 5998–6008.
14. Yadav, S.; Tan, Z.-H. Audio Mamba: Selective State Spaces for Self-Supervised Audio Representations. *arXiv* **2024**, arXiv:2406.02178.
15. Gao, X.; Chen, N.F. Speech-Mamba: Long-Context Speech Recognition with Selective State Spaces Models. In *Proceedings of the 2024 IEEE Spoken Language Technology Workshop (SLT), Macau, China, 2–5 December 2024*; IEEE: Piscataway, NJ, USA, 2024; pp. 1–8.
16. Shams, S.; Dindar, S.S.; Jiang, X.; Mesgarani, N. SSAMBA: Self-Supervised Audio Representation Learning with Mamba State Space Model. *arXiv* **2024**, arXiv:2405.11831.
17. Deng, J.; Guo, J.; Xue, N.; Zafeiriou, S. ArcFace: Additive angular margin loss for deep face recognition. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), Long Beach, CA, USA, 15–20 June 2019*; pp. 4690–4699.
18. Wang, F.; Cheng, J.; Liu, W.; Liu, H. Additive margin softmax for face verification. *IEEE Signal Process. Lett.* **2018**, *25*, 926–930. [\[CrossRef\]](#)

19. Thiyagarajan, S.; Kim, D.-H. A data-oriented conceptual model and hybrid TDNN–BiLSTM framework for context-aware speaker verification in smart environments. In Proceedings of the 2026 IEEE International Conference on Big Data and Smart Computing (BigComp), Guangzhou, China, 2–5 February 2026; pp. 394–397.
20. Thiyagarajan, S.; Kim, D.-H. Symmetry and asymmetry principles in deep speaker verification systems: Balancing robustness and discrimination through hybrid neural architectures. *Symmetry* **2026**, *18*, 121. [[CrossRef](#)]

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.